

puzzle8-astar

Nguyen An Loc

41052 Advanced Algorithms, University of Technology Sydney
Programming Assignment 1 (Track B)

1 Introduction

The 8-puzzle is a 3×3 board with tiles 1 to 8 and one empty square, the blank. A move slides a tile next to the blank into the blank. The goal is to reach the board in Figure 1(a), with the tiles in order and the blank in the bottom-right corner, in as few moves as possible. Only half of the $9! = 362,880$ arrangements can reach the goal, so 181,440 boards are solvable, and the hardest need 31 moves. I write a board as its 9 digits read row by row, with 0 for the blank, so the goal is 123456780. A solution is a string of U, D, L and R: the direction the blank moves.

Finding a shortest solution is a shortest-path problem on a graph whose nodes are boards and whose edges are moves. This project solves it with A* search, and compares two heuristics that guide the search.

2 What I built

I implemented A* search for the 8-puzzle in Python with two heuristics: Manhattan distance, and Manhattan distance plus linear conflict. Both are admissible and consistent. An inversion-parity check rejects unsolvable inputs before any search runs.

To check the solver, I built an *oracle*: the true fewest-moves distance of every solvable board. It runs breadth-first search (BFS) outward from the goal, like ripples in water: level 1 holds every board one move away, level 2 every board two moves away, and so on up to level 31. Because every move can be undone, this one BFS gives the distance of all 181,440 solvable boards. The oracle shares no code with A*, so A* is never checked against itself. It checks both heuristics on every state, and every A* path in the tests and in `compare` must match its distance.

I chose the 8-puzzle over the 15-puzzle. The 15-puzzle has $\sim 10^{13}$ states, too many for exhaustive ground truth, and plain A* needs pattern-database heuristics to stay tractable there. A fully verified 8-puzzle solver is better than a partly verified 15-puzzle one.

How the two heuristics work

A* explores states in the order $f(s) = g(s) + h(s)$, where $g(s)$ is the number of moves made to reach s and $h(s)$ is an estimate of the moves still needed. If h never overestimates (admissible), the first time A* pops the goal, its path is optimal. If h also changes by at most 1 per move (consistent), A* never needs to revisit a state. Among admissible heuristics, a larger h is a closer estimate, so fewer states look worth exploring.

Manhattan distance. One move slides one tile by one square. So each tile needs at least as many moves as its row distance plus column distance to its home square, and h is the sum over all tiles, not counting the blank. On board (b) in Figure 1, this gives 14, while the true distance is 20.

Linear conflict. Manhattan treats tiles as if they could slide through each other, but they cannot. If two tiles are both in their goal row but in reversed order, one of them must step out of the row and back in, which costs at least 2 moves that Manhattan does not count. Linear

conflict adds these 2-move penalties in every row and column. On board (c), tiles 1 and 2 are reversed in the top row, and so are 4 and 5 in the middle row. Manhattan gives 4, linear conflict gives $4 + 2 + 2 = 8$, and the true distance is 16. Because linear conflict only adds to Manhattan, it is never below it.

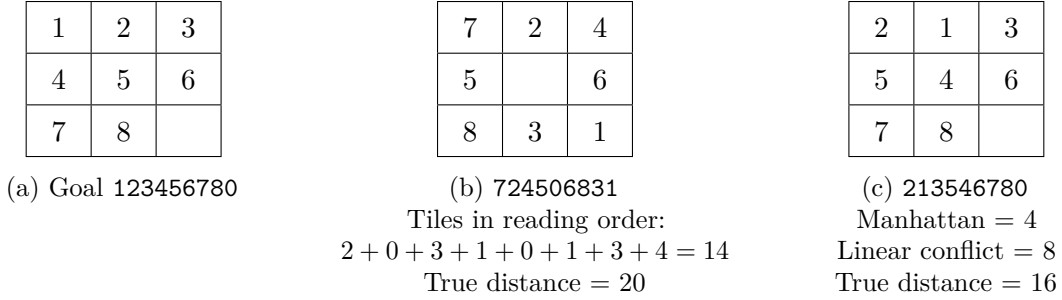


Figure 1: The goal and two example boards. The empty square is the blank.

3 The tool

`python -m puzzle8` has 3 commands. `solve` takes `--heuristic` and `--stats`. `random --depth D` returns a state whose depth the oracle confirms, not a random walk's length. `compare` runs the benchmark below. Exit codes separate solved, unsolvable, bad input, and internal error, and no raw traceback ever reaches the user.

Figure 2 shows where A* sits inside `solve`. Bad or unsolvable input is rejected before any search runs, and every path A* returns is replayed from the start state before it is printed.

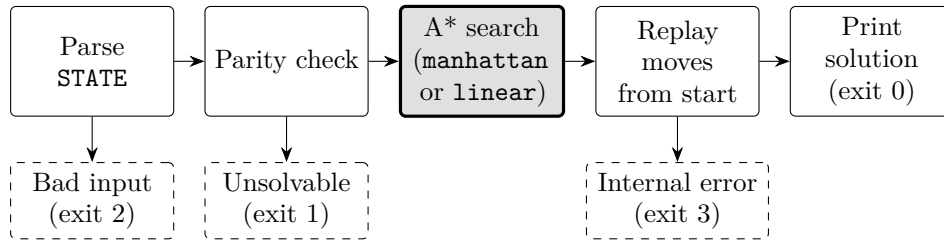


Figure 2: How `solve` handles one input. A* (shaded) is the only step that searches.

1. **Parse.** `parse_state` accepts 9 digits, or spaced or comma-separated digits if quoted. A non-digit (even ²), a wrong tile count, a missing blank, a tile above 8 or a duplicate tile gets a one-line error (exit code 2).
2. **Parity check.** Read the 8 tiles row by row, ignoring the blank. An *inversion* is a pair of tiles in the wrong order, such as 2 before 1 in 213456780. `is_solvable` counts them. Every move changes the count by 0 or 2 and the goal has 0, so an odd count is unsolvable (exit code 1). Without it, A* would expand all 181,440 reachable states first.
3. **A* search.** `astar` uses a heap ordered by $(f, h, \text{insertion order})$, a goal test on pop and a closed set, and returns the moves and the `--stats` counts.
4. **Replay.** `apply_moves` in `board.py` plays the moves from the start state, and the result must be the goal. A mismatch is a search bug, so the tool prints only a one-line internal error (exit code 3).
5. **Print.** Moves are the direction the blank moves (U/D/L/R). `--stats` adds node counts, the largest frontier and the time; `--show-boards` prints every board.

Interface decisions. Search, replay and output share one move table in `board.py`. `random` prints only the state, so `solve "$(python3 -m puzzle8 random --depth 12)"` works. Exit codes let a script tell “no solution exists” (1) from “you typed it wrong” (2). Unexpected exceptions print one `internal error` line, never a traceback.

One of the two hardest states, 31 moves from the goal:

```
$ python3 -m puzzle8 solve 867254301 --stats
Start:      867254301
Heuristic:  manhattan
Solution:   31 moves (optimal)
Moves:      LURDRUULDLDRRUULLDDRRULULDDRURD
            (U/D/L/R = the direction the blank moves)

Nodes expanded:  6744
Nodes generated: 10396
Max frontier:    3549
Time:           14.52 ms
```

An unsolvable state stops at step 2:

```
$ python3 -m puzzle8 solve 213456780
Unsolvable: 213456780 has 1 inversion (odd).
Every move keeps the inversion count's parity, and the goal has 0,
so only states with an even count can reach it. No search was run.
```

Search work is measured in *nodes expanded*: the number of boards A* pops from the heap and generates neighbours for. `compare` solves the same sampled states with both heuristics. In a run with 200 states/depth bucket, the median per-state ratio of nodes expanded (Manhattan over linear conflict) was:

Table 1: Median per-state ratio of nodes expanded, Manhattan over linear conflict.

Bucket	Depths	Median ratio	Interquartile range
Shallow	5–10	1.00	1.00–1.00
Medium	15–20	1.63	1.40–1.88
Deep	21–24	1.96	1.66–2.29
Hard	25–31	1.91	1.79–2.09

Linear conflict expanded more nodes than Manhattan on 3/800 states.

4 What I learned

A stronger heuristic can make A* do more work. Linear conflict is never below Manhattan, yet in 12 of 3,000 uniformly sampled states it expanded more nodes: fewer in 2,943, tied in 45, and more in 12. The state with the largest surplus, 760584312 (26 moves from the goal), took 846 expansions against Manhattan’s 685.

The cause is tie-breaking order, not a weak heuristic. To see why, split the states by their f value, where C^* is the optimal solution cost (26 on that state). A* pops states in order of increasing f , and the goal comes off the heap with $f = C^*$. With a consistent heuristic, every state with $f < C^*$ is therefore expanded before the goal. A* has no choice about these, so I call them the *forced set*. States with $f > C^*$ are never expanded. That leaves the states with f exactly equal to C^* , the *final plateau*. They tie with the goal, so whether A* expands one of them before popping the goal depends only on how it breaks ties. My solver breaks ties toward lower h , then first-in, first-out.

Since linear conflict is never below Manhattan, any state with $f < C^*$ under linear conflict also has $f < C^*$ under Manhattan. So linear conflict’s forced set is a subset of Manhattan’s: 224 states against 515 on that state.

The plateau is where the result gets surprising. In all 12 states, linear conflict’s plateau was also smaller: 3,067 states against Manhattan’s 5,146 in total, and 761 against 1,331 on that state. Yet, pooled over the 12 states, A* expanded 77% of the linear conflict plateau and only 16% of the Manhattan plateau. On that state, that was 622/761 against 170/1,331 (Figure 3). The stronger heuristic shrank both the forced set and the plateau, then lost inside the plateau. A test in the suite checks the smaller plateau and the tie-only surplus on each of the 12 states.

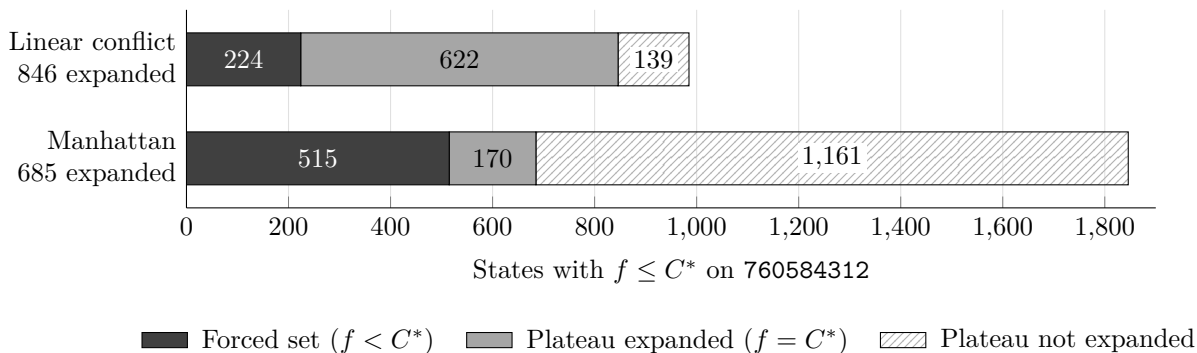


Figure 3: Where the expansions go on 760584312. Linear conflict has the smaller forced set and the smaller plateau, but A* expands most of its plateau before popping the goal.

So, nodes expanded, the usual measure of heuristic quality, partly measures tie-breaking rather than the heuristic itself. The forced-set size does not depend on tie-breaking, and it measures the heuristic alone.

Fukunaga (2026) studies this final plateau directly. For a consistent heuristic and nine standard tie-breaking strategies, every pair has instances where either strategy expands arbitrarily more states than the other. My experiment fixes the tie-breaking strategy (lower h , then first-in first-out) and varies the heuristic instead, so it is not an instance of that theorem, but a related result from the other side: once the forced set is done, the expansion count depends on the plateau and the order A* searches it.

5 AI use

Which tools I used: I used Claude Code as my primary AI tool. Opus 5 wrote most of the implementation, across roughly 9 stages. I reviewed and stopped after every stage before the next one was built on it. Besides, I used Grammarly to correct grammar in this report.

What I used it for: Claude to scaffold each module from a specification I wrote (board representation, heuristics, BFS oracle, the search, the CLI, the benchmark), to write a test suite, to draft this report, and to convert it to L^AT_EX for the best demonstration.

Two specific examples of where it was wrong:

1. The parser first used `str.isdigit()`, which also accepts non-ASCII digit-like characters such as ² or Arabic-Indic digits. It either crashed with the wrong exception or silently parsed a different board than the one that was typed. I restricted the check to ASCII 0–9 and added a regression test for both cases.
2. While drafting this report, Claude claimed linear conflict “pushed far more states to exactly $f = C^*$.” The plateau sizes show the opposite: linear conflict’s plateau is smaller, but far

more of it gets expanded. I asked for the raw numbers instead of the summary, checked that the forced set plus plateau-expanded matched the total I'd already measured, and added the corrected claim as a test assertion.

What I understood versus what I took on trust: I learned the basics of A* search on graphs and trees in SC3000 Artificial Intelligence at NTU last semester, and this project was my first chance to implement it, on the 8-puzzle. I understand why the heap is ordered by $f = g + h$, why the goal test runs when the goal is popped, and what the closed set and lazy deletion do. I can explain why Manhattan never overestimates, why linear conflict counts the tiles that must leave a line (using the longest increasing subsequence rather than conflicting pairs), why the inversion-parity rule leaves only half of the $9!$ boards reachable ($9!/2 = 181,440$), and why the BFS oracle can check A* independently. I also understand the forced-set and plateau result in Section 4, and why “nodes expanded” partly measures tie-breaking.

I took the consistency proof for linear conflict on trust: I did not derive it myself. Instead, I rely on the exhaustive test over all 181,440 reachable states. That test covers every state of the 8-puzzle, but it does not explain why consistency holds, and it says nothing about larger boards. I also trusted Claude Code to write the test suite. I understood the exhaustive tests, but I did not review all 147 tests line by line. Lastly, Opus wrote most of the comments and docstrings as part of code generation. I trimmed and rephrased some of them, but I did not read all of them, especially in the `tests/` folder.

To avoid relying on trust alone, every path the tool prints is replayed from the start state, and every path in the tests and in `compare` is also checked against the oracle's optimal length. Both heuristics are checked on all 181,440 states, not on samples. Before submitting, I asked for the raw numbers and had every figure in this report re-run from the code.

References

Fukunaga, A. (2026). *Expansion Counts under Standard A* Tie-Breaking Strategies on the Final Plateau*. arXiv:2609.23293.